

Discussion Globex FedEx — Q&R

Export du 23/06/2026. Version PDF abrégée — le fichier Word contient l'intégralité des réponses.

Question 1

lis bien mon projet d'abord en sachant que le projet principal est globex-fedex ou est integre jarvis juste pour ne pas interopre mais je crois que tu sais deja ca

Réponse : Exploration de la structure du projet pour clarifier l'intégration entre globex-fedex et Jarvis. [REDACTED] Oui — j'ai parcouru le dépôt et la structure correspond bien à ce que tu décris. Voici ma lecture consolidée. Vue d'ensemble Le repo `jarvis-OS` est un monorepo avec **deux mondes distincts** : | Zone | Chemin | Rôle | |---|---|---| | **Projet principal** | `fedex-main/globex-fedex-main/` | Plateforme **Globex FedEx** (métier logistique) | | **Sidecar intégré** | `src/jarvis/` + config racine | **Jarvis OS** / **Fedex-v0** — assistant IA personnel, branché sans casser Globex | Globex **ne dépend pas** de Jarvis pour tourner seul. Jarvis est un **satellite** : UI admin, voix, mémoire, dashboard workspace, pont vers l'agent Globex. --- Globex FedEx (cœur métier) **Stack** : **FastAPI** + **PostgreSQL** + **Angular 17**. **Trois portails** (routes Angular) : - **Client** — `/chat`, tracking, documents, support, notifications - **Employé** — `/employee/` (clients, colis, tickets, assistant IA interne) - **Admin** — `/admin` (command center, sécurité, missions agent, KPI, etc.) **Backend** (`backend/app/`) : - Routes métier : auth, chat, tracking FedEx, support, rapports, notifications... - **Globex Agent** (`/api/globex-agent/`) — noyau IA **indépendant** du copilote admin, basé sur Ollama + outils métier (exports, users, tickets, SLA...) - **Jarvis bridge** (`services/jarvis/`) — client HTTP vers Jarvis pour l'**Agent Window** admin - Migration en cours vers le nom **fedex-v0** (même rôle que Jarvis côté sidecar) **Port par défaut** : `8000` (cf. `api.config.ts` frontend). --- Jarvis OS (intégration non intrusive) **Stack** : **assistant personnel** (FastAPI, mémoire SQLite, voix LiveKit, mission engine, outils Gmail/Spotify/vision...). **Chez toi** (`.env` racine) : - Jarvis tourne sur `PORT=8013` - `GLOBEX_OS_ENABLED=true` → le chat UI Jarvis **ne parle plus au LLM local seul**, il proxy vers Globex **Ponts d'intégration** : `mermaid flowchart LR subgraph globex [Globex FedEx : 8000] GA[Globex Agent API] AW[Agent Window admin] end subgraph jarvis [Jarvis / Fedex-v0 : 8013] UI[UI home.js] PX[/api/globex/chat] WS[/api/workspace/globex/] end UI --> PX --> GA WS --> GA AW --> jarvis/bridge.py UI AdminAngular --> openJarvisUi UI 1. Jarvis → Globex : src/jarvis/interfaces/api/globex.py appelle /api/globex-agent/chat avec login admin (GLOBEX_ADMIN_EMAIL/PASSWORD). 2. Globex → Jarvis : bo... [Suite dans le fichier Word]`

Question 2

Tu m'as aidé à développer mon projet Globex FedEx : chatbot intelligent de suivi de colis avec frontend Angular, backend FastAPI, base PostgreSQL, API FedEx, export Excel/PDF, authentification, historique, notifications et interface admin. Je veux maintenant préparer mon rapport de stage/PFE complet...

Réponse : Analyse exhaustive du dépôt pour produire un rapport structuré basé uniquement sur le code vérifié. [REDACTED] Rapport d'analyse technique — Globex FedEx (PFE / stage) > Analyse réalisée **uniquement** par lecture du dépôt (`fedex-main/globex-fedex-main/`), sans modification ni exécution des tests. Les statuts **réalisé / partiel / non réalisé** sont déduits du code source. --- 1. Résumé global du projet | Élément | Contenu vérifié | |---|---| | **Nom du projet** | **Globex FedEx** — API nommée `Globex FedEx Chatbot API`, version `2.4.0` (`backend/app/core/config.py`) | | **Objectif principal** | Plateforme web de suivi de colis FedEx assistée par un chatbot IA, avec authentification multi-rôles, historique, exports et console d'administration | | **Problème résolu** | Centraliser le suivi FedEx, la conversation assistée et la gestion opérationnelle (clients, employés, admin) dans une seule application | | **Utilisateurs ciblés** | **Clients** (`role=client`), **employés** (`role=employee`), **administrateurs** (`role=admin`) — enum `UserRole` dans `backend/app/models/user.py` | | **Résultat final obtenu** | Application **fonctionnelle en local** : frontend Angular 17, backend FastAPI, PostgreSQL, intégration FedEx (sandbox + mode mock), chatbot multi-LLM (Gemini/Ollama), admin avancé, notifications, exports Excel/PDF, sécurité JWT/2FA/IDS | | **État actuel** | **Partiellement prêt pour production** : cœur métier **réalisé** ; déploiement production, FedEx production et certaines intégrations **partielles ou non finalisées** | | **Synthèse par bloc** : | Bloc | Statut | |---|---| | Chatbot client + tracking | **Réalisé** (HTTP, pas WebSocket client) | | API FedEx | **Partiel** (`FEDEX_ENABLED=false` par défaut → mock ; sandbox avec liste blanche) | | Auth / rôles / admin | **Réalisé** | | Exports Excel/PDF | **Réalisé** | | Multilingue FR/AR/EN | **Réalisé** (UI + prompts + exports) | | Déploiement production | **Non**

réalisé** (seul Docker PGVector dev) | Extensions (employé, Globex Agent, Jarvis) | **Réalisées dans le code**, au-delà du CDC initial | --- 2. Comparaison avec le cahier des charges initial | Fonctionnalité CDC | Réalisé dans le projet | Fichiers principaux | Statut | Remarque technique | ---|---|---|---|---| Chatbot temps réel | Chat HTTP POST `/chat/message` ; réponse synchrone | `routes/chat.py`, `services/chatbot\_service.py`, `frontend/.../chatbot.service.ts` | **Partiel** | Pas de WebSocket côté client ; « temps réel... [Suite dans le fichier Word]

Question 3

Donne-moi uniquement la liste complète des endpoints FastAPI avec méthode, URL, rôle, fichier, fonction, auth requise et exemple de réponse

Réponse : Extraction exhaustive de tous les endpoints FastAPI depuis les fichiers de routes. [REDACTED] Liste exhaustive des endpoints **montés dans `backend/app/main.py`** (plus sous-routeurs inclus dans `admin.py`). **Base URL : `http://127.0.0.1:8000`** Auth : **`Authorization: Bearer <JWT>`** sauf mention « Public ». > **Doublons de préfixe : certains routeurs sont montés deux fois** (`/support` + `/api/support`, `/admin/settings` + `/api/settings`, etc.). Les URLs ci-dessous indiquent le chemin canonique ; la variante `/api/...` est équivalente quand indiquée. --- Racine | Méthode | URL | Rôle | Fichier | Fonction | Auth | Exemple réponse | ---|---|---|---|---| GET | `/` | Healthcheck API | `main.py` | `root` | Public | `{"status":"ok","service":"Globex FedEx Chatbot API"}` | GET | `/api/dashboard/stats` | KPI dashboard admin | `main.py` | `api\_dashboard\_stats` | admin | `{"users\_total":120,"sessions\_today":45,...}` | --- Authentification --- `/auth/\*` (routes/auth.py) | Méthode | URL | Rôle | Fonction | Auth | Exemple réponse | ---|---|---|---|---| POST | `/auth/register` | Inscription client | `register` | Public | `{"id":1,"email":"user@mail.com","role":"client",...}` | POST | `/auth/register/employee` | Inscription employé (pending) | `register\_employee` | Public | `{"detail":"Votre demande...","email":"...","status":"pending"}` | POST | `/auth/activate` | Activation employé (token email) | `activate\_account` | Public | `UserRead` | POST | `/auth/login` | Connexion | `login` | Public | `{"access\_token":"eyJ...","requires\_2fa":false}` ou `{"requires\_2fa":true,"challenge\_token":"..."}` | GET | `/auth/google/config` | Config OAuth Google | `google\_auth\_config` | Public | `{"enabled":false,"client\_id":null}` | POST | `/auth/google` | Login Google | `login\_with\_google` | Public | `{"access\_token":"eyJ...","token\_type":"bearer"}` | POST | `/auth/2fa/verify` | Valider OTP 2FA | `verify\_2fa` | Public | `{"access\_token":"eyJ...","token\_type":"bearer"}` | POST | `/auth/2fa/resend` | Renvoyer OTP | `resend\_2fa` | Public | `{"detail":"Code renvoyé."}` | GET | `/auth/me/qrcode` | Payload QR login | `get\_qr\_login\_payload` | JWT | `{"token":"...","expires\_at":"..."}` | POST | `/auth/me/qrcode/regenerate` | Régénérer QR | `regenerate\_qr\_login` | JWT | idem | POST | `/auth/qrcode` | Connexion via QR | `qr\_login` | Public | `Token` | GET | `/auth/me` | Profil courant | `read\_me` | JWT | `{"id":1,"full\_name":...` [Suite dans le fichier Word]

Question 4

Donne-moi uniquement la structure complète de la base PostgreSQL : tables, colonnes, types, relations, clés étrangères, et diagramme Mermaid ERD. et tt type de relation et ce qui va me rendre infos dans ce contexte

Réponse : Lecture de tous les modèles SQLAlchemy pour documenter la base PostgreSQL avec précision. [REDACTED] Structure PostgreSQL — Globex FedEx **Base : `globex\_fedex`** (`DATABASE\_URL` dans `backend/app/core/config.py`) **ORM : SQLAlchemy** — modèles dans `backend/app/models/`, création via `init\_db()` (`backend/app/core/database.py`) **Extension optionnelle : `vector` (PGVector) sur `knowledge\_chunks.embedding` — colonne `vector(768)`** **Total : 45 tables** identifiées dans le code. --- Légende des types PostgreSQL | Type SQLAlchemy | Type PostgreSQL | ---|---| `Integer` (PK auto) | `SERIAL` / `INTEGER` | `String(n)` | `VARCHAR(n)` | `Text` | `TEXT` | `Boolean` | `BOOLEAN` | `DateTime(timezone=True)` | `TIMESTAMPZ` | `Float` | `DOUBLE PRECISION` | `Float` (nullable) | `DOUBLE PRECISION` NULL | `vector(768)` | `vector(768)` *(PGVector, optionnel)* | Légende des relations | Symbole | Signification | ---|---| **PK** | Clé primaire | **FK → table(col)** | Clé étrangère avec `ON DELETE` | **UNIQUE** | Contrainte d'unicité | **INDEX** | Index (non unique) | **1:N** | Un parent, plusieurs enfants | **1:1** | Relation un-à-un | **Logique** | Lien métier sans FK SQL | --- 1. Noyau utilisateurs et authentification `users` — Comptes (client / employé / admin) **Fichier : `models/user.py`** Colonne | Type PG | Contraintes | Rôle | ---|---|---| `id` | SERIAL | **PK** | Identifiant | `full\_name` | VARCHAR(255) | NOT NULL | Nom affiché | `email` | VARCHAR(255) | NOT NULL, **UNIQUE** | INDEX | Login | `password\_hash` | VARCHAR(255) | NOT NULL | Mot de passe bcrypt | `role` | VARCHAR(32) | NOT NULL, défaut `client` | `client` | \ `employe` | \ `admin` | `status` | VARCHAR(20) | NOT NULL, défaut `active` | `pending` | \ `invited` | \ `active` | \ `suspended` | `activation\_token` | VARCHAR(96) | UNIQUE, NULL | Token activation employé | `activation\_expires\_at` | TIMESTAMPTZ | NULL | Expiration activation | `preferred\_language` | VARCHAR(8) | NOT NULL, défaut `fr` | Langue UI | `response\_preferences` | TEXT | NOT NULL, défaut `` | Préférences réponse IA | `preference\_profile\_json` | TEXT | NOT

NULL, défaut '{}'; Profil préférences structuré | `quota\_messages\_per\_day` | INTEGER | NULL | Quota messages | `quota\_trackings\_per\_day` | INTEGER | NULL | Quota trackings | `quota\_exports\_per\_day` | INTEGER | NULL | Quota exports | `organization\_id` | VARCHAR(64) | NOT NULL, **UNIQUE**, IND... [Suite dans le fichier Word]

Question 5

Donne-moi uniquement la liste des composants Angular, routes, pages, services utilisés, API appelées et captures à mettre dans le rapport.

[illegible]

Question 6

Donne-moi tous les modules admin existants, ce qu'ils font, les fichiers concernés et ce qui est encore incomplet.

Réponse : Exploration du code admin pour lister les modules, leurs rôles, les fichiers associés et les parties incomplètes. [REDACTED] Voici l'inventaire des ****modules admin**** du projet Globex FedEx ('fedex-main/globex-fedex-main'), basé sur le code actuel. --- Architecture commune | Élément | Rôle | Fichiers | ---|---|---| ****Shell admin**** | Layout, sidebar, recherche globale, cloche, ouverture Jarvis externe, routing interne |

`frontend/src/app/features/admin/admin-page.component.{ts,html,scss}` | ****Route**** | `/admin` (+ enfants
`/admin/agent-missions/\*`) | `frontend/src/app/app.routes.ts`, `frontend/src/app/core/guards/admin.guard.ts` | ****Drawers**
transverses** | Tickets support client + chat employé | `admin-support-drawer/`, `admin-employee-chat-drawer/` | ****Services**
partagés** | `admin.service.ts`, `command-center.service.ts`, `admin-ai.service.ts`, `notifications.service.ts`,
`system-settings.service.ts`, `reports.service.ts`, `gpt-knowledge.service.ts`, `agent-missions.service.ts`,
`agent-window.service.ts`, `admin-conversations.service.ts` | `frontend/src/app/core/services/` | ****Backend agrégé**** |
Routes | `admin/\*`, `api/admin/\*`, `api/settings/\*`, `api/reports/\*` | `backend/app/routes/admin.py`, `security\_admin.py`,
`settings\_admin.py`, `notifications.py`, `reports.py`, `gpt\_knowledge.py`, `agent\_missions.py`, `agent\_window.py`,
`support.py` | ****Sections déclarées mais sans entrée sidebar** : ****`analytics`, `monitoring`** (alias du dashboard), **`roles`** (filtre
employés uniquement), **file `preferences`** (voir module 14). --- 1. Dashboard (Command Center) ****Fonction** : **** Vue exécutive**
— KPIs, graphiques ngx-charts, expéditions live, conversations récentes, santé système, mini-copilot IA intégré, liens de
navigation vers les autres modules. | Couche | Fichiers | ---|---| Frontend |

`components/admin-dashboard/admin-dashboard.component.{ts,html,scss}` | Service | `command-center.service.ts`,
`admin-ai.service.ts` (requête IA dashboard) | Backend | `admin.py` → `GET /admin/command-center`, `POST
/admin/command-center/ai` ; service `command center service.py` | **\*\*Incomplet / partiel\*\*** : Sections **`analytics`** et

`monitoring` réutilisent le même composant sans vue distincte. - Dépend fortement de données agrégées backend ; pas de fallback demo explicite (erreur affichée si API indisponible). --- 2. Assistant IA admin (Copilot) **Fonction** : Chat copilot admin, pièces jointes (PDF/image), exports (logs, rapports, tracking), comma... [Suite dans le fichier Word]

Question 7

Donne-moi un plan de tests complet avec cas de test, étapes, résultat attendu et résultat obtenu pour chaque fonctionnalité.

Réponse : Exploration du dépôt pour recenser les fonctionnalités et les tests existants, afin de produire un plan de tests réaliste. [REDACTED] Plan de tests structuré pour **Globex FedEx v2.4.0** (Angular + FastAPI + PostgreSQL). Les **résultats obtenus** ne sont pas remplis ici : aucune campagne n'a été exécutée dans cette session. La colonne indique **comment les renseigner** et signale les **écarts connus dans le code**. --- 0. Prérequis de campagne | Élément | Valeur |
|---|---| Backend | `http://127.0.0.1:8000` | Frontend | `http://127.0.0.1:4200` | Base | PostgreSQL `globex\_fedex` |
Comptes | 1 client, 1 employé validé, 1 admin | FedEx | Sandbox (`FEDEX\_ENABLED=true`) + numéro whitelist | LLM | Gemini ou Ollama configuré dans `.env` | Jarvis (optionnel) | Port `8013` si tests sidecar | **Types de test** : **M** = Manuel (UI) - **A** = Automatisé backend (`backend/tests/`) - **P** = Postman/cURL **Légende résultat obtenu** : **`[À EXÉCUTER]`** — à remplir après test - **`[AUTO]`** — couvert par pytest existant (lancer `pytest backend/tests/`) - **`[PARTIEL]`** — fonctionnalité incomplète vérifiée dans le code --- 1. Authentification & comptes | ID | Cas | Type | Étapes | Résultat attendu | Résultat obtenu |
|---|---|---|---|---|---| AUTH-01 | Inscription client | M | `/register` → remplir formulaire → soumettre | Compte créé, redirection login ou message succès | **`[À EXÉCUTER]`** | AUTH-02 | Login client | M/P | `POST /auth/login` ou UI `/login` | JWT retourné, redirection `/chat` | **`[À EXÉCUTER]`** | AUTH-03 | Login employé | M | `/employee/login` avec compte `employee` | Redirection `/employee` | **`[À EXÉCUTER]`** | AUTH-04 | Inscription employé | M | `/employee/register` | Statut `pending`, message activation | **`[À EXÉCUTER]`** | AUTH-05 | Activation employé | M | Lien `/employee/activate?token=...` | Compte `active`, login possible | **`[À EXÉCUTER]`** | AUTH-06 | Login admin | M | Compte `role=admin` → `/admin` | Accès shell admin, guard OK | **`[À EXÉCUTER]`** | AUTH-07 | Guard client sur `/chat` | M | Accès `/chat` sans token | Redirection `/login` | **`[À EXÉCUTER]`** | AUTH-08 | Guard admin | M | Client tente `/admin` | Accès refusé / redirect | **`[À EXÉCUTER]`** | AUTH-09 | Profil `GET /auth/me` | P | Token client → GET | Profil JSON complet | **`[À EXÉCUTER]`** | AUTH-10 | Mise à jour profil | M | `/settings/profile` → modifier nom → sauver | PATCH `/auth/me` OK, UI à jour | **`[À EXÉCUTER]`** | AUTH-11 | 2FA (si activée) | M | Login → sais... [Suite dans le fichier Word]

Question 8

Donne-moi les difficultés techniques rencontrées dans le projet et les solutions appliquées, avec les fichiers concernés

Réponse : Exploration du dépôt pour identifier les difficultés techniques documentées dans le code, les guides et les tests. [REDACTED] Synthèse des **difficultés techniques** identifiées dans le dépôt et des **solutions appliquées**, avec les fichiers concernés. Les éléments proviennent du code, des tests et de la doc (`docs/prompt-security.md`, guides). --- 1. Évolution du schéma PostgreSQL sans Alembic **Difficulté** : Le projet a grossi (45+ tables, colonnes ajoutées en cours de dev) sans migrations versionnées Alembic — risque de schémas incohérents entre environnements. **Solution** : `create\_all()` au démarrage + **~71 ALTER/CREATE idempotents** (`IF NOT EXISTS`) dans `init\_db()`. **Fichiers** : - `backend/app/core/database.py` - `backend/app/models/\*.py` --- 2. Intégration FedEx sandbox (numéros limités) **Difficulté** : L'API FedEx sandbox n'accepte qu'un sous-ensemble de numéros de test ; les appels avec numéros arbitraires échouent ou polluent les démos. **Solution** : Liste blanche centralisée + blocage avant appel API. - Message utilisateur explicite `sandbox\_whitelist\_denied`. - Affichage frontend dédié sur les cartes colis. **Fichiers** : - `backend/app/services/fedex\_sandbox\_whitelist.py` - `backend/app/services/fedex\_service.py` → `assert\_sandbox\_whitelist()` - `backend/app/services/client\_agent\_service.py` - `backend/app/services/llm/prompt\_assembly.py` - `frontend/src/app/shared/utils/shipment-card.util.ts` - `frontend/src/app/core/services/tracking.service.ts` → `isSandboxWhitelistError()` - `frontend/src/app/core/i18n/translations/fr.ts`, `en.ts` --- 3. Hallucinations IA sur le statut des colis **Difficulté** : Le LLM inventait statuts/ETA quand FedEx ne renvoyait rien ou en cas d'erreur API. **Solution** : Réponse **anti-hallucination** si données FedEx indisponibles (`llm\_provider: none`). - Mode optionnel **réponse déterministe** depuis l'API (`USE\_DETERMINISTIC\_FEDEX\_REPLY`) sans repasser par le LLM. - Champ `source` sur les messages (`fedex\_api`, `fallback`, etc.). **Fichiers** : - `backend/app/services/llm/orchestrator.py` → `\_anti\_hallucination\_track\_reply()`, `\_format\_fedex\_deterministic\_reply()` - `backend/app/core/config.py` - `backend/.env.example` - `docs/prompt-security.md` - `backend/app/services/chatbot\_service.py` - `frontend/src/app/core/services/chat-session.service.ts` --- 4. Injections de

prompt et préférences utilisateur risquées ****Difficulté**** : Les utilisateurs peuvent tenter d'injecter ... [Suite dans le fichier Word]

Question 9

Donne-moi les textes prêts à coller dans mon rapport pour le chapitre Réalisation, avec un style académique." "Donne-moi les diagrammes Mermaid pour architecture globale, cas d'utilisation, séquence chatbot, séquence tracking, séquence export, et base de données."

Réponse : Vérification de quelques points d'architecture dans le code pour garantir l'exactitude des textes et des diagrammes. [REDACTED] Voici un contenu ****prêt à coller**** pour le chapitre ****Réalisation****, suivi des ****diagrammes Mermaid****. Le style est académique (formulations impersonnelles, vocabulaire PFE). Les éléments correspondent à l'implémentation réelle du dépôt ****Globex FedEx Chatbot v2.4.0**** (`fedex-main/globex-fedex-main`). --- Chapitre 5 — Réalisation 5.1 Introduction Le présent chapitre expose la mise en œuvre concrète de la plateforme ****Globex FedEx****, assistant conversationnel orienté logistique et suivi de colis. La réalisation s'appuie sur une architecture ****three-tier**** : une interface utilisateur développée en ****Angular 17****, une API métier en ****FastAPI**** (Python) et une base de données relationnelle ****PostgreSQL****. L'objectif fonctionnel est de permettre à trois profils — client, employé et administrateur — d'interagir avec un chatbot intelligent, de consulter des expéditions via l'API FedEx, de gérer des tickets de support et, côté administration, de superviser l'activité de la plateforme au moyen d'un centre de commandement et d'outils d'assistance artificielle. La réalisation a été conduite de manière itérative : un noyau métier (authentification, chat, tracking) a d'abord été stabilisé, puis enrichi par des modules transverses (exports, notifications, sécurité, agents autonomes) et par une intégration ****sidecar**** avec ****Jarvis-OS****, sans remplacement du backend principal. --- 5.2 Environnement technique et outils Le développement a été réalisé dans un environnement local standardisé. Le backend Python s'exécute sur le port ****8000**** ; le frontend Angular sur le port ****4200****. La configuration est externalisée dans des fichiers `.env` (modèle fourni par `.env.example`). La persistance repose sur PostgreSQL (`globex_fedex`). Les principaux outils retenus sont les suivants : | Couche | Technologies | |---|---| | Frontend | Angular 17, TypeScript, RxJS, SCSS, ngx-charts | | Backend | FastAPI, SQLAlchemy, Pydantic, httpx | | Base de données | PostgreSQL (+ extension vector pour la RAG lorsque disponible) | | IA | Google Gemini, Ollama (repli), moteur déterministe local | | Intégration | API FedEx Track (sandbox/production), Jarvis-OS (port 8013) | | Tests backend | pytest (~50 fichiers de tests unitaires et d'intégration) | L'initialisation du schéma est assurée au démarrage par `create_all()` complété de migrations SQL idempotentes... [Suite dans le fichier Word]

Question 10

y'a des infos qui reste qui vont nous servir ?

Réponse : Oui — il reste du matériel utile que nous n'avons pas encore livré sous forme de ****bloc prêt à coller****, même si une grande partie du rapport est déjà couverte. Déjà disponible dans nos échanges | Contenu | Statut | |---|---| | Chapitre ****Réalisation**** (texte académique) | Livré | | ****6 diagrammes Mermaid**** (archi, UC, séquences, BDD) | Livré | | ****Modules admin**** (rôle, fichiers, incomplet) | Livré | | ****Plan de tests**** (~140 cas, colonne « obtenu » à remplir) | Livré | | ****Difficultés techniques + solutions + fichiers**** | Livré | | Endpoints FastAPI (~230 URLs) | Livré (conversation précédente) | | Structure PostgreSQL (45 tables) | Livré (conversation précédente) | | Rapport PFE 19 sections (analyse globale) | Livré (conversation précédente) | --- Ce qui reste utile et ****pas encore livré en bloc dédié**** 1. Inventaire frontend complet (priorité rapport) Demandé plus tôt, pas encore sorti en une seule liste structurée : - ****84 composants Angular**** (client / employé / admin / shared) - ****Routes + pages**** (`app.routes.ts` + agent-missions) - ****29 services**** + API par service - ****Captures d'écran à prévoir**** avec légendes (client, employé, admin, auth, Postman/Swagger) → Indispensable pour les chapitres ****Conception IHM****, ****Réalisation frontend**** et ****Annexes****. 2. Catalogue de captures d'écran Le plan de tests mentionne les cas, mais pas encore un ****tableau Figure X.X**** du type : - capture | URL | prérequis | légende académique → À faire pour la partie ****Tests**** et ****Présentation****. 3. Chapitres textuels prêts à coller (autres que Réalisation) On peut encore produire, sur le même modèle académique : - ****Introduction / Contexte / Problématique**** - ****État de l'art**** - ****Analyse des besoins / CDC**** - ****Conception**** (architecture + BDD + IHM) - ****Tests et validation**** (résultats à remplir après exécution) - ****Conclusion et perspectives**** - ****Annexes**** (liste endpoints, DDL, comptes de test) 4. Résultats de tests réels Le plan de tests est prêt, mais la colonne ****« Résultat obtenu »**** est vide — il faut lancer backend + campagne manuelle (ou mode Agent pour exécuter `pytest`). 5. Tableau « Réalisé / Partiel / Non réalisé » Utile pour la ****Conclusion**** et la ****Discussion**** : - composer conversations admin sans API → ****Partiel**** - file préférences sans menu → ****Partiel**** - agent-window non monté → ****Partiel**** - tests frontend → ****Non réalisé**** - etc. 6. Guide de déploiement / démo soutenance Pas encore formalisé : -

ordr... [Suite dans le fichier Word]

Question 11

tu peux me faire tous cette discussion dans un document word ou pdf et tu le genere pour moi a telechrger

Réponse : Compilation de la discussion en un document téléchargeable. Vérification des outils disponibles, puis génération du fichier. [REDACTED] Création d'un script Python pour générer le document Word complet, puis conversion en PDF si possible. [REDACTED] Les documents ont été générés. Voici où les trouver : Fichiers à télécharger | Fichier | Chemin | Taille | Contenu | |--|--|--|--| | **Word (complet)** |
`fedex-main/globex-fedex-main/docs/RAPPORT\_PFE\_GLOBEX\_FEDEX\_DISCUSSION.docx` | ~42 Ko | Document principal avec tout le détail | | **PDF (synthèse)** |
`fedex-main/globex-fedex-main/docs/RAPPORT\_PFE\_GLOBEX\_FEDEX\_DISCUSSION.pdf` | ~3 Ko | Résumé ; pour un PDF complet, exporter depuis Word | Chemins absolus : - `C:\Users\HP\Downloads\jarvis-OS\fedex-main\globex-fedex-main\docs\RAPPORT\_PFE\_GLOBEX\_FEDEX\_DISCUSSION.docx` - `C:\Users\HP\Downloads\jarvis-OS\fedex-main\globex-fedex-main\docs\RAPPORT\_PFE\_GLOBEX\_FEDEX\_DISCUSSION.pdf` Contenu du document Word (11 sections) 1. Contexte et stack technique 2. Chapitre **Réalisation** (texte académique) 3. Architecture et diagrammes Mermaid 4. Inventaire frontend (routes, services, composants) 5. Modules administrateur + statuts 6. Plan de tests (extrait + ~140 cas) 7. Difficultés techniques et solutions 8. Captures d'écran recommandées (14 figures) 9. Guide démarrage et maintenance 10. Synthèse réalisé / partiel / non réalisé 11. Annexes (backend, BDD, tests pytest) Régénérer les fichiers Un script est disponible si tu modifies le contenu : ``bash python fedex-main/globex-fedex-main/docs/generate\_rapport\_discussion.py `` PDF complet Ouvre le `.docx` dans Word → **Fichier** → Enregistrer sous → PDF pour obtenir un PDF avec la mise en forme complète (tableaux, sections, etc.). Si tu veux, je peux enrichir le PDF automatique pour qu'il reprenne l'intégralité du Word (version plus longue). [REDACTED]

Question 12

non mais je veux cette discussion avec qst et reponses tel qu'ils sont pas de rapport

Réponse : Compris — tu voulais la **conversation brute** (questions + réponses telles quelles), pas une synthèse type rapport. J'ai généré un export depuis le transcript Cursor de notre discussion : **Fichiers :** -
`docs/DISCUSSION\_QR\_GLOBEX\_FEDEX.docx` (~100 Ko) — **document principal** avec les 12 échanges Q/R complets -
`docs/DISCUSSION\_QR\_GLOBEX\_FEDEX.pdf` — version PDF **abrégée** (réponses tronquées ; pour le PDF intégral : ouvrir le .docx → Enregistrer sous → PDF) **Contenu :** chaque échange est structuré en **Question N** puis **Réponse**, avec le texte original des réponses (rapport PFE 19 sections, endpoints, PostgreSQL, Angular, modules admin, plan de tests, difficultés techniques, chapitre Réalisation + Mermaid, etc.). **Script de régénération :** ``bash python
fedex-main/globex-fedex-main/docs/generate_conversation_qr.py `` L'ancien fichier
`RAPPORT\_PFE\_GLOBEX\_FEDEX\_DISCUSSION.docx` était une synthèse rapport — celui-ci est bien la discussion Q/R.